MODULE 05 · FOUNDATIONS - ALL DOMAINS

Kanban, Lean, XP and scaling agile

Flow instead of timeboxes, waste and pull, the engineering practices behind quality, and what changes when many teams share one product.

DOMAIN Foundations EXAM WEIGHT All domains MODULE 05 of 49

admin@sthithapragna.com

Scrum manages work in timeboxes.
Kanban manages **the flow** of work.

Lean thinking in five ideas

- **Value** is defined by the customer. Everything else is a candidate for removal.
- The **value stream** is every step from request to delivery, including the waiting.
- **Flow**: work should move steadily, without stopping, batching or looping back.
- **Pull**: start work when there is capacity downstream, not because it arrived.
- **Continuous improvement**, done with respect for the people doing the work.

> ON THE EXAM
>
> Lean asks one question of every step: does the customer value this? If not, shrink it or remove it.

Seven wastes in knowledge work

WASTE	LOOKS LIKE
Partially done work	Code written but not tested; designs never built
Extra features	Functions nobody asked for, or nobody uses
Relearning	Solving the same problem twice because nothing was shared
Handoffs	Knowledge lost each time work passes to another group
Delays	Waiting for approvals, decisions, environments
Task switching	People spread across many items, finishing few
Defects	Rework, and the cost of finding problems late

ON THE EXAM

Delays and task switching are the wastes most scenarios describe. Both are fixed by limiting work in progress.

Value stream mapping

- Draw every step from request to delivery, with the **working time** and the **waiting time** for each.
- Done with the people involved in each step, not from a desk.
- Typical finding: a few days of work spread across weeks of waiting between steps.
- Improve the **biggest wait** first. Working faster rarely helps if the work is mostly waiting.

> ON THE EXAM
>
> If a scenario says work takes days but delivery takes weeks, map the value stream before changing anything.

A simple Kanban board

01
Options
Ideas and requests. No limit.

›

02
Ready
Refined and agreed. Limit 5.

›

03
Doing
In progress. Limit 3.

›

04
Review
Checked by a peer. Limit 2.

›

05
Done
Meets the agreed policy.

ON THE EXAM

The numbers are WIP limits. A full column means nobody starts new work there; they help finish.

Kanban's core practices

PRACTICE	WHAT IT MEANS
Visualise the work	Every item and every step is on the board
Limit work in progress	Explicit limits per column, or for the whole system
Manage flow	Watch how items move; remove blockages quickly
Make policies explicit	Write down the rules, such as what 'done' means for each column
Implement feedback loops	Regular meetings to review flow, service and risks
Improve collaboratively	Evolve the system in small, agreed steps

ON THE EXAM

Kanban starts with what you do now and improves it. No new roles are required on day one.

A motorway at 70% capacity flows. At 100% it stops.

Little's Law: the arithmetic of flow

- **Average cycle time = average work in progress / average throughput.**
- 12 items in progress, finishing 4 a week: each item takes about 3 weeks.
- Halve the WIP to 6, same throughput: about 1.5 weeks. Same people, faster delivery.
- Adding work without adding capacity only makes every item slower.

> ON THE EXAM
>
> To deliver faster without more people, reduce work in progress. The arithmetic shows why.

Flow metrics

METRIC	MEASURES	USEFUL FOR
Lead time	Request to delivery	What the customer experiences
Cycle time	Work starts to work finishes	How efficient the team's process is
Throughput	Items finished per period	Forecasting how much will be delivered
Work in progress	Items started but not finished	Spotting overload early

ON THE EXAM

Lead time includes the wait before work starts. Customers feel lead time; teams control cycle time.

Scrum or Kanban?

SCRUM

Timeboxed batches

- Fixed-length sprints
- Defined accountabilities
- Commits to a sprint goal
- Measures velocity per sprint

KANBAN

Continuous flow

- No iterations; pull as capacity frees
- No required roles
- Commits through WIP limits
- Measures cycle time and throughput

THE TIEBREAKER

Predictable product work that benefits from a planning rhythm: Scrum. Unpredictable arrival and urgent items: Kanban.

Extreme programming: the engineering practices

PRACTICE	WHY IT HELPS
Pair programming	Two people, one task: fewer defects, shared knowledge
Test-driven development	Write the test first, then the code to pass it
Continuous integration	Merge and test many times a day; problems surface early
Refactoring	Improve the design of working code without changing behaviour
Small releases	Get working software to users often
Collective ownership	Anyone can improve any part; no single point of failure

ON THE EXAM

When a scenario describes late integration surprises or recurring defects, XP practices are usually the fix.

Spikes: buying knowledge

- A **spike** is a short, timeboxed piece of research or prototyping to reduce uncertainty.
- Use it when the team cannot estimate or choose an approach, because it lacks knowledge.
- The output is knowledge, a prototype or a decision, not a finished feature.
- Not an excuse for open-ended research. The timebox is the point.

> ON THE EXAM
>
> Estimates that vary wildly usually mean missing knowledge. A spike buys that knowledge cheaply.

Agile-adjacent: design thinking and Lean Startup

- **Design thinking**: empathy with users, and cheap prototypes that invite feedback while change is still cheap.
- **Lean Startup**: treat each idea as a hypothesis. Build, measure, learn, then persevere or pivot.
- A **minimum viable product** here is an experiment: the smallest thing that tests the hypothesis.
- The Agile Practice Guide, Second Edition, treats these as complementary disciplines, not agile methods.

> ON THE EXAM
>
> Unsure whether anyone wants it? Test the idea cheaply before building it properly.

Other names you may meet

NAME	REMEMBER IT FOR
Scrumban	Scrum's rhythm combined with Kanban's flow and WIP limits
DSDM	Fixed time and cost, flexible features; the origin of MoSCoW prioritisation
Crystal	A family of methods scaled to team size and criticality
Feature-driven development	Building and tracking work feature by feature

ON THE EXAM

You will not be tested on the details. Recognise the name and its one big idea.

Scaling: many teams, one product

- The core problem is **dependencies and integration** between teams.
- **Scrum of Scrums**: representatives meet regularly to coordinate cross-team work and blockers.
- Frameworks: SAFe, LeSS, Scrum@Scale, and PMI's own Disciplined Agile.
- Common ingredients: one product backlog, aligned cadences, shared definition of done, continuous integration.

> ON THE EXAM
>
> The exam tests the problem, not the framework. Cross-team dependencies need a coordination mechanism.

Project Lantern: more than one agile style

- The **app team** uses Scrum, with XP practices: continuous integration and test-driven development.
- The **go-live support team** uses Kanban, with an expedite lane for clinic-stopping faults.
- A weekly **coordination meeting** links the app team, the kiosk supplier and the data migration team.
- Each team works the way that suits its work. The project manager integrates them.

> ON THE EXAM
>
> Different work, different approach, one project. Integration is the project manager's job.

Matching the approach to the work

THE SCENARIO DESCRIBES	REACH FOR
Product features, planned in batches, with regular stakeholder reviews	Scrum
Unpredictable arrivals, urgent interruptions, support or operations	Kanban
Recurring defects, late integration problems, fragile code	XP practices
An idea nobody is sure customers want	Lean Startup, an MVP experiment
Work that takes days of effort but weeks to deliver	Value stream mapping
Several teams breaking each other's work	Scaling: coordination, shared DoD, CI

ON THE EXAM

Name the problem first. The right practice follows from the problem, not from a favourite framework.

> An IT support team of five uses a Kanban board. Customers complain that requests take weeks, even simple ones.
>
> The board shows eighteen items in the In Progress column. Team members say they are busy all day but rarely finish anything.

What should the project manager do?

A Facilitate the team in setting work-in-progress limits for its board columns.

B Request two more team members to handle the volume.

C Move the team to two-week sprints with a sprint goal.

D Ask each team member to work on the oldest items first.

◆ PAUSE AND DECIDE

What should the project manager do?

A Facilitate setting WIP limits.

Eighteen items for five people is the cause. Limits force finishing, so cycle time drops.

B Request more people.

Adds cost without fixing the overload. They would also start too much.

C Move to sprints.

Changes the framework without fixing the actual problem, too much in progress.

D Oldest first.

A sensible policy, but it does not stop people starting new work.

THE SIGNAL

Busy all day, finishing little, many items open: too much work in progress.

> A team tracks its flow. Over the last quarter it has averaged 20 items in progress at any time, and finished an average of 5 items per week.
>
> The product owner asks how long a newly started item will typically take to finish.

What is the best estimate of the average cycle time?

A 1 week

B 4 weeks

C 15 weeks

D 25 weeks

◆ PAUSE AND DECIDE

What is the best estimate of the average cycle time?

A 1 week

That would need throughput of 20 a week, or WIP of 5.

B 4 weeks

Little's Law: 20 items in progress divided by 5 finished per week.

C 15 weeks

Subtracts instead of divides. The law is a ratio.

D 25 weeks

Adds the numbers. It has no meaning as a time.

THE SIGNAL

Cycle time = WIP / throughput. Check the units: items divided by items per week gives weeks.

> During refinement, developers estimate a story to connect the app to a payment provider none of them has used before.
>
> Estimates range from 3 to 40 points, and nobody can explain the difference with confidence.

What should the team do?

A Run a short, timeboxed spike to investigate the provider, then estimate again.

B Use the average of the estimates so planning can continue.

C Use the highest estimate to be safe.

D Ask the product owner to set the estimate.

◆ PAUSE AND DECIDE

What should the team do?

A Timeboxed spike, then re-estimate.

The spread comes from missing knowledge. A spike buys it cheaply.

B Average the estimates.

Averages ignorance. The number looks precise and means nothing.

C Take the highest.

Pads the plan without reducing the uncertainty at all.

D Product owner sets it.

Estimates belong to the people doing the work, not the product owner.

THE SIGNAL

Wildly different estimates plus unfamiliar technology: a knowledge gap. Spike it.

> In a hospital's facilities department, small change requests take about two days of actual work, but six weeks from request to completion.
>
> Each request passes through four departments for sign-off. Staff in every department say they are working flat out.

What should the project manager do first?

A Add another technician to complete the work faster.

B Map the value stream with the four departments to find where requests wait.

C Ask each department head to approve requests more quickly.

D Buy a workflow tool to automate the approvals.

◆ PAUSE AND DECIDE

What should the project manager do first?

A. Add a technician.

The work is two days. Speeding up the work barely touches six weeks of waiting.

B. Map the value stream together.

Rungs 2 and 3: finds where the waiting is, with the people who own each step.

C. Ask for faster approvals.

Pressure without understanding. Everyone is already busy, so nothing changes.

D. Buy a workflow tool.

A solution before the diagnosis. It may automate the wrong steps.

THE SIGNAL

Days of work, weeks of elapsed time: the problem is waiting. Find it before fixing anything.

> Four Scrum teams build one online banking product. Each team's increment passes its own tests.
>
> At every sprint review, features from different teams conflict with each other, and fixing them takes most of the next sprint.

What should the project manager do?

A Let each team release its own features independently.

B Assign a coordinator to each team to report progress daily to the project manager.

C Add a separate integration sprint after every three development sprints.

D Help the teams set up regular cross-team coordination, a shared definition of done and continuous integration.

◆ PAUSE AND DECIDE

What should the project manager do?

A Release independently.

The features interact in one product. Separate releases push conflicts onto customers.

B Daily reports to the PM.

Adds reporting, not integration. The conflicts still appear late.

C Integration sprints.

Batches integration, so conflicts pile up for weeks before anyone sees them.

D Coordination, shared DoD, continuous integration.

Surfaces conflicts daily instead of every sprint, at the source.

THE SIGNAL

Teams fine alone, broken together: an integration problem. Integrate continuously and coordinate across teams.

> An infrastructure operations team adopted two-week sprints. Urgent incidents arrive unpredictably and must be handled within hours.
>
> Almost every sprint is disrupted, planned improvement work rarely finishes, and the team finds sprint planning pointless.

What should the project manager recommend?

A Lengthen the sprints to four weeks so there is more room for incidents.

B Stop planning, and handle work in whatever order it arrives.

C Move to Kanban, with an expedite lane for incidents and WIP limits for improvement work.

D Plan all improvement work in a predictive schedule for the year.

◆ PAUSE AND DECIDE

What should the project manager recommend?

A Four-week sprints.

More time to be disrupted. Incidents still arrive unpredictably.

B No planning at all.

Loses control entirely; improvement work would never be protected.

C Kanban with an expedite lane and WIP limits.

Handles urgent arrivals explicitly while still protecting and finishing planned work.

D A yearly predictive plan.

Even less able to absorb interruptions than two-week sprints.

THE SIGNAL

Unpredictable, urgent arrivals break timeboxes. Flow with explicit policies absorbs them.

Stop starting, **start finishing**. Then choose the practice that fits the problem.